Day 4, Session 2: Advanced FSMs - Pattern Detection & Mealy Machines

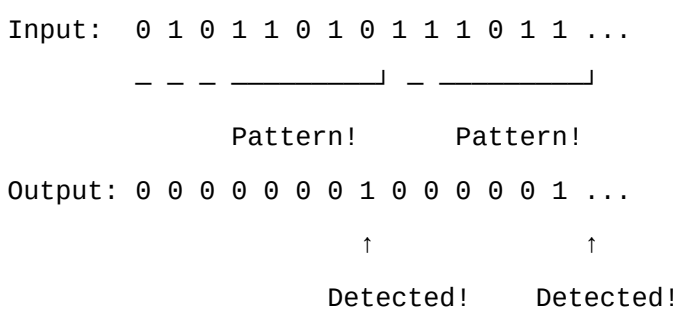
---

Concept Block (12 min)

Sequence Detector - The Classic FSM Problem

Problem: Design a circuit that detects the pattern **"1011"** in a serial input stream.

Example input stream:



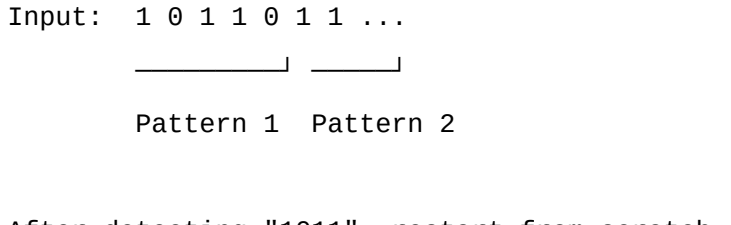
This is used in:

- Frame synchronization (networking)
- Start-of-packet detection (UART, USB)
- Pattern recognition (bioinformatics, signal processing)
- Error detection codes

Two Approaches: Overlapping vs Non-Overlapping

Non-Overlapping Detection:

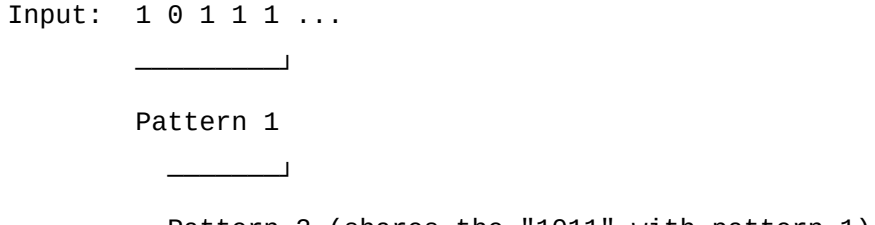
Patterns **cannot share bits**.



After detecting "1011", restart from scratch.

Overlapping Detection:

Patterns **can share bits**.



`After detecting "1011", check if the last bits can start a new pattern.`

We'll implement overlapping detection (more complex, more useful).

State Diagram Design Process

Step-by-step design for "1011" detector:

States needed:

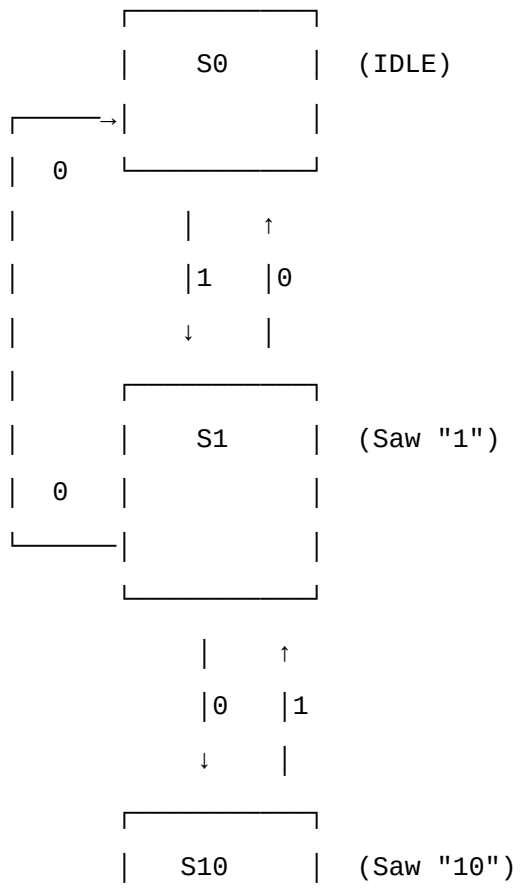
1. **IDLE**: No pattern progress (or just saw 0)
2. **S1**: Saw "1"
3. **S10**: Saw "10"
4. **S101**: Saw "101"
5. **S1011**: Saw "1011" - **DETECTED!** (but immediately check if we can continue)

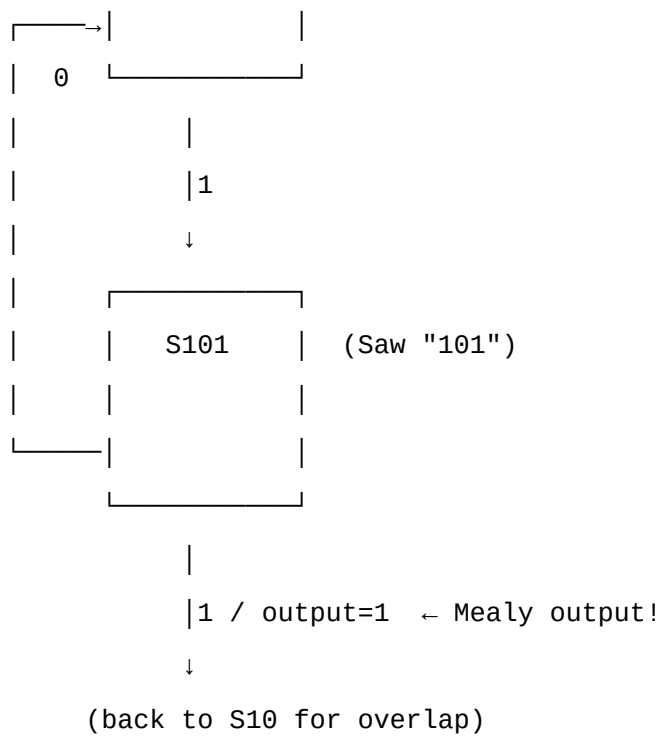
Actually, we can optimize to 4 states:

1. **S0**: Idle / No progress
2. **S1**: Saw "1"
3. **S10**: Saw "10"
4. **S101**: Saw "101"

When we see the 4th bit (1), we detect the pattern and can transition directly.

State Diagram: Sequence Detector (Mealy)





Key Mealy Feature: Output=1 happens **on the transition**, not in a state!

Mealy vs Moore for Sequence Detection

Mealy Approach (What we'll build):

```
// Output depends on current_state AND input
if (current_state == S101 && in == 1)
    detected = 1;  // Output immediately
```

Advantages:

- Fewer states needed
- Faster response (output on same cycle as detection)
- More compact

Disadvantages:

- Output can glitch if input changes
- Harder to verify

Moore Approach (Alternative):

```
// Need an extra "DETECTED" state
// Output depends only on state

States: S0, S1, S10, S101, S1011_DETECTED
// Output=1 only in S1011_DETECTED state
```

Advantages:

- Glitch-free outputs
- Easier to verify
- More intuitive

Disadvantages:

- Requires extra state
- 1 clock cycle delay in output

Code & Practical Block (15 min)

Implementation: Sequence Detector (Mealy Machine)

```
// sequence_detector.v - Detects "1011" pattern (Mealy FSM)
module sequence_detector (
    input  wire clk,
    input  wire rst,
    input  wire in,         // Serial input stream
    output reg  detected    // 1 when "1011" detected
);

    // State encoding
    localparam S0   = 2'b00;  // Idle / No progress
    localparam S1   = 2'b01;  // Saw "1"
    localparam S10  = 2'b10;  // Saw "10"
    localparam S101 = 2'b11;  // Saw "101"

    // State registers
    reg [1:0] current_state;
    reg [1:0] next_state;

    // Block 1: State Register (Sequential)
    always @(posedge clk or posedge rst) begin
        if (rst)
            current_state <= S0;
        else
            current_state <= next_state;
    end

    // Block 2: Next State Logic (Combinational)
```

```
    always @(*) begin
        case (current_state)
            S0: begin
                if (in)
                    next_state = S1;    // Saw first "1"
                else
                    next_state = S0;    // Stay in idle
            end

            S1: begin
                if (in)
                    next_state = S1;    // Another "1", restart pattern
                else
                    next_state = S10;   // Saw "10"
            end

            S10: begin
                if (in)
                    next_state = S101;  // Saw "101"
                else
                    next_state = S0;    // Broke pattern, restart
            end

            S101: begin
                if (in)
                    next_state = S10;   // Saw "1011"! Overlap: "10" of next
pattern
                else
                    next_state = S10;   // Saw "1010", continue with "10"
            end

            default: next_state = S0;
        endcase
    end

    // Block 3: Output Logic (Combinational - MEALY)
    always @(*) begin
```

```
        // Mealy: Output depends on current_state AND input
        if (current_state == S101 && in == 1)
            detected = 1;       // Pattern "1011" detected!
        else
            detected = 0;
    end

endmodule
```

Line-by-Line: Key Differences from Moore

Output Logic (Mealy - Lines 62-68):

```
always @(*) begin
    if (current_state == S101 && in == 1)  // ← Depends on INPUT!
        detected = 1;
    else
        detected = 0;
end
```

Critical difference from Moore:

- **Moore:** `if (current_state == DETECTED_STATE)`
- **Mealy:** `if (current_state == S101 && in == 1)`

Mealy output changes IMMEDIATELY when input changes (combinational path from input to output).

Overlapping Detection (Lines 50-53):

```
S101: begin
    if (in)
        next_state = S10;   // Detected "1011", but "10" starts next pattern!
    else
        next_state = S10;   // Saw "1010", have "10" for next attempt
end
```

Why transition to S10?

```
Input stream: ...1 0 1 1 1...
              ──────────┘ Pattern detected!
                    ──┘ These bits ("10") start checking for next pattern
```

Without overlap, we'd go back to S0 and miss patterns like:

```
Input: 1 0 1 1 0 1 1
```

```
       ┌─────────┘     ← Pattern 1
           ┌─────────┘ ← Pattern 2 (overlaps!)
```

Testbench: Sequence Detector

```
// tb_sequence_detector.v
module tb_sequence_detector;

    reg  clk, rst, in;
    wire detected;

    sequence_detector uut (
        .clk(clk),
        .rst(rst),
        .in(in),
        .detected(detected)
    );

    // Clock generator
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Test stimulus
    initial begin
        $dumpfile("sequence_detector.vcd");
        $dumpvars(0, tb_sequence_detector);

        $display("Time | in | detected | State | Pattern Progress");
        $display("-----------------------------------------------");

        // Initialize
        rst = 1; in = 0;
        #10;
        rst = 0;
        #10;
```

```
// Test sequence: 0 1 0 1 1 0 1 0 1 1 1
//                      ┘     ┘
//                      Pattern!   Overlapping pattern!

$display("\n--- Input: 0 (idle) ---");
in = 0; #10;

$display("\n--- Input: 1 (start pattern) ---");
in = 1; #10;

$display("\n--- Input: 0 (now have '10') ---");
in = 0; #10;

$display("\n--- Input: 1 (now have '101') ---");
in = 1; #10;

$display("\n--- Input: 1 (DETECT '1011'!) ---");
in = 1; #10;

$display("\n--- Input: 0 (break pattern) ---");
in = 0; #10;

$display("\n--- Input: 1 (restart) ---");
in = 1; #10;

$display("\n--- Input: 0 (have '10') ---");
in = 0; #10;

$display("\n--- Input: 1 (have '101') ---");
in = 1; #10;

$display("\n--- Input: 1 (DETECT '1011' again!) ---");
in = 1; #10;

$display("\n--- Input: 1 (have '101' from overlap) ---");
```

```
        in = 1; #10;

        $display("\n--- Input: 1 (DETECT '1011' third time!) ---");
        // Current state is S10 (from "11"), next "1" gives S101, then "1"
detects!
        in = 1; #10;

        #20;
        $finish;
    end

    // Monitor state changes
    always @(posedge clk) begin
        $display("%4t | %b  |    %b     |  %s   | %s",
                 $time, in, detected,
                 (uut.current_state == 2'b00) ? "S0  " :
                 (uut.current_state == 2'b01) ? "S1  " :
                 (uut.current_state == 2'b10) ? "S10 " :
                 (uut.current_state == 2'b11) ? "S101" : "??? ",
                 (uut.current_state == 2'b00) ? "Idle" :
                 (uut.current_state == 2'b01) ? "Saw: 1" :
                 (uut.current_state == 2'b10) ? "Saw: 10" :
                 (uut.current_state == 2'b11) ? "Saw: 101" : "Unknown");
    end

endmodule
```

Expected Behavior

```
Time | in | detected | State | Pattern Progress
------------------------------------------------
--- Input: 0 (idle) ---
  25 | 0  |    0     |  S0   | Idle

--- Input: 1 (start pattern) ---
  35 | 1  |    0     |  S1   | Saw: 1
```

```
--- Input: 0 (now have '10') ---
  45 | 0  |    0     |  S10  | Saw: 10

--- Input: 1 (now have '101') ---
  55 | 1  |    0     |  S101 | Saw: 101

--- Input: 1 (DETECT '1011'!) ---
  65 | 1  |    1     |  S10  | DETECTED! ✅ (now have '10' for overlap)

--- Input: 0 (break pattern) ---
  75 | 0  |    0     |  S0   | Idle

--- Input: 1 (restart) ---
  85 | 1  |    0     |  S1   | Saw: 1

--- Input: 0 (have '10') ---
  95 | 0  |    0     |  S10  | Saw: 10

--- Input: 1 (have '101') ---
 105 | 1  |    0     |  S101 | Saw: 101

--- Input: 1 (DETECT '1011' again!) ---
 115 | 1  |    1     |  S10  | DETECTED! ✅
```

Compile and Test

```
iverilog -o sim/seq_det sequence_detector.v tb_sequence_detector.v
vvp sim/seq_det
gtkwave sequence_detector.vcd
```

Mini Task (10 min)

Build a "101" sequence detector (simpler pattern):

Requirements:

1. Detect pattern "101"
2. Use Mealy machine (output on transition)
3. Support overlapping detection
4. Use 3-block coding style

States needed:

- S0: Idle
- S1: Saw "1"
- S10: Saw "10"
- (No S101 state - detect on transition!)

Template:

```
module seq_101_detector (
    input  wire clk,
    input  wire rst,
    input  wire in,
    output reg  detected
);

    localparam S0  = 2'b00;
    localparam S1  = 2'b01;
    localparam S10 = 2'b10;

    reg [1:0] current_state, next_state;

    // Block 1: State register
    always @(posedge clk or posedge rst) begin
        // TODO
    end

    // Block 2: Next state logic
    always @(*) begin
        // TODO
    end

    // Block 3: Output logic (MEALY!)
    always @(*) begin
        // TODO: When should detected=1?
        // Hint: current_state == S10 && in == ???
    end

endmodule
```
